

Project 7: Stream Analysis

Algorithms for Massive Datasets

Master in Computer Science - Università degli Studi di Milano, 2025/26

Gian Marco Bratzu

June 2026

Abstract

This report describes the implementation of three streaming algorithms applied to the *New York Times Articles & Comments 2020* dataset. The comment log is interpreted as a time-ordered stream, and three algorithms are implemented from scratch: **Flajolet-Martin** (count of distinct users), **Alon-Matias-Szegedy** (second moment of the section distribution), and a **Bloom filter** (set-membership test for articles belonging to a given section). Each estimator is compared against the computed exact value to measure the approximation error. All three work in a single pass and use memory independent of the stream length. The full code is available at: <https://github.com/Bratzu93/Algorithms-for-massive-datasets>.

1 Introduction

Streaming algorithms address the scenario in which data arrives as a continuous sequence of elements that can be processed only once, without the ability to store the entire stream. The fundamental constraints are one-pass processing and sublinear (ideally constant) memory usage. In exchange, only approximate answers are provided, with provable probabilistic guarantees.

This project applies three classical streaming techniques to user comments from The New York Times, treating the sequence of comments, ordered by publication timestamp as the stream. The three problems addressed are:

1. Counting **distinct users** (Flajolet-Martin), specifically the 0th frequency moment F_0 .
2. Estimating the **second moment** of the comment distribution across sections (Alon-Matias-Szegedy): how much does traffic concentrate on a few sections?
3. **Bloom filter** for set membership: given an article ID, is it in Opinion?

2 Dataset

Source

New York Times Articles & Comments (2020), Kaggle, identifier `benjaminawd/new-york-times-articles-comm` (CC BY-NC-SA 4.0, accessed June 2026) [2].

Files

- `nyt-articles-2020.csv`: 16,787 articles with fields including `uniqueID`, `section`, `headline`, `pub_date`. The 42 distinct sections include U.S. (2,364 articles), Opinion (2,272), World (1,183), Arts (1,094), and New York (1,055).

- `nyt-comments-part0.csv`: first partition of the comments, approximately 300 MB and 500,000 rows. Used for the subsampled run.
- `nyt-comments-2020.csv`: single pre-merged file of approximately 3 GB containing **4,986,461 comments** in total. Used for the full-scale run. Relevant fields (both files): `userID` (integer), `createDate` (timestamp), `articleID` (URI string).

Download

The dataset is downloaded via the Kaggle CLI at notebook startup. On Google Colab the credentials (`KAGGLE_USERNAME`, `KAGGLE_KEY`) are read from Colab Secrets; locally from `~/.kaggle/kaggle.json`. The dataset is not committed to the repository.

3 Data Organisation and Pre-processing

Stream construction

The stream is the sequence of comments sorted in ascending order by `createDate`, since the timestamp format is `YYYY-MM-DD HH:MM:SS`.

Section enrichment

The comments file does not carry section information; it is added via a pandas left join on the articles file, matching `comments.articleID` to `articles.uniqueID`. The articles file is loaded with only the two columns needed (`uniqueID`, `section`) and merged into the comments DataFrame in a single pass; `uniqueID` is then dropped, leaving a `section` column alongside each comment. Manual inspection confirmed that every comment matches an article in the articles file, so no NaN values appear in the `section` column after the join.

Subsampling

A global boolean variable `SUBSAMPLE` controls data size at runtime.

- `SUBSAMPLE = True` (default): loads `nyt-comments-part0.csv`, sorts by timestamp, and retains the first 200,000 comments. This setting is designed for Google Colab and completes in a few minutes.
- `SUBSAMPLE = False`: loads `nyt-comments-2020.csv` directly and sorts the full 4,986,461-comment stream. Intended for local execution.

Stream simulation

The stream is simulated by sorting the DataFrame by `createDate` and iterating over the elements with a `for` loop, passing one item at a time to each algorithm’s `update` method, mirroring how a real stream would trigger updates on arrival. The full dataset is held in memory as input, but the streaming constraint applies to the *algorithm state*: FM stores H integers and AMS stores k counters, both independent of the stream length. The Bloom filter stores m bits, where m is computed from the number of elements to insert n and the target FPR; its memory therefore grows with the positive set size, not with the stream length. One acknowledged limitation is that AMS requires knowing n before the stream begins in order to sample positions uniformly; in a truly online setting this would require a preliminary counting pass.

4 Algorithms and Implementations

4.1 Flajolet-Martin (Distinct Elements)

The goal is to estimate $F_0 = |\{x : f_x > 0\}|$, the number of distinct `userID` values, in a single pass over the stream.

Algorithm

For a hash function h that maps elements uniformly over $\{0, 1\}^b$, the probability that $h(x)$ ends in at least k trailing zero bits is 2^{-k} . Consequently, if R is the maximum number of trailing zeros observed across all elements in the stream, then $\hat{F}_0 = 2^R$ is an estimate of F_0 .

To reduce variance, H independent hash functions are used. Their $R_{\max}$ values are divided into G groups; within each group the median is taken (to suppress outliers arising from $h(x) = 0$, which would give an infinite tail length), and the final estimate is the mean of the per-group estimates:

$$\hat{F}_0 = \frac{1}{G} \sum_{g=1}^G 2^{\text{median}(R^{(g)})}$$

Initial approach: MD5 with sequential salts

A first implementation derived H hash functions from MD5 by varying a numeric salt:

$$h_i(x) = \text{MD5}(\text{str}(x) + \text{"_"} + \text{str}(\text{seed} + i)) \bmod 2^{128}, \quad i = 0, 1, \dots, H - 1$$

The trailing zero count was read by converting the integer output to its binary string representation and iterating over characters from the right. This approach was replaced for two reasons. First, **performance**: the MD5 version required approximately 30 seconds on the sub-sample and 20 minutes on the full dataset, versus 3 seconds and 4 minutes respectively for the universal hash implementation — a roughly 5–10 $\times$ slowdown attributable to string conversion and the character-by-character trailing-zero loop. Second, **implementation**: the bit-trick `(h & -h).bit_length() - 1` computes the trailing-zero count in $O(1)$ without any string parsing, and hash functions of the form $(ax + b) \bmod p$ provide stronger theoretical independence guarantees than MD5 with sequential salts.

Final implementation: universal hashing

Each hash function is a member of the 2-universal family:

$$h_i(x) = (a_i \cdot x + b_i) \bmod p, \quad p = 2^{61} - 1 \text{ (Mersenne prime)}$$

with $a_i \in [1, p - 1]$ and $b_i \in [0, p - 1]$ chosen independently at random. Coefficients sampled from a space of $\sim 2^{61}$ values ensure statistical independence across the H hash functions, enabling genuine variance reduction.

Trailing zeros

The number of trailing zero bits of h is computed with the identity:

`(h & -h).bit_length() - 1`

The expression `h & -h` isolates the least significant set bit (e.g. $12_{10} = 1100_2 \rightarrow 0100_2 = 4$); `bit_length() - 1` gives its position, which equals the trailing-zero count. I used `FM_NUM_HASHES = 64` and `FM_NUM_GROUPS = 8`.

Results

H	MD5		Universal hash	
	Estimate	Error	Estimate	Error
8	55 296	8.7%	266 240	423.3%
16	53 745	5.6%	69 426	36.5%
32	45 347	10.9%	96 281	89.2%
64	61 028	20.0%	53 042	4.3%
128	54 945	8.0%	48 449	4.8%
256	56 144	10.4%	59 041	16.0%
512	57 344	12.7%	63 137	24.1%

Table 1: FM on the subsample (200,000 comments, $G = 8$ groups, $\text{SEED}=1$, exact $F_0^* = 50,876$). Run time for $H = 256$ –512: MD5 2m 40s, universal hash 33s.

H	MD5		Universal hash	
	Estimate	Error	Estimate	Error
8	1 933 312	379.7%	663 552	64.6%
16	992 155	146.2%	534 568	32.6%
32	409 118	1.5%	465 056	15.4%
64	501 800	24.5%	540 190	34.0%
128	499 471	23.9%	374 021	7.2%

Table 2: FM on the full dataset (4,986,461 comments, $G = 8$ groups, $\text{SEED}=1$, exact $F_0^* = 403,025$). Run time: MD5 20m 10s, universal hash 4m 20s.

Universal hashing is approximately $5\times$ faster than MD5 in both settings.

This illustrates a key limitation of single-run evaluation: a single extreme $R_{\max}$ value can shift a group median regardless of how many hash functions are used, making the error non-monotone in H . The formula gives the expected error over many independent runs, not a per-run guarantee.

The number of groups G controls the confidence level: a larger G reduces the probability that all group medians are simultaneously wrong. For robust estimates, averaging multiple independent runs (each with a different seed) is more reliable than relying on a single run with a large H .

4.2 Alon-Matias-Szegedy (Second Moment)

The second frequency moment $F_2 = \sum_j m_j^2$ (where m_j is the number of comments in section j) measures how concentrated the distribution is across sections: the more uneven, the higher F_2 .

Algorithm

Assume the stream length n is known in advance. Sample k positions $p_1, \dots, p_k$ uniformly at random from $[0, n)$. For each sampled position p_i , maintain a variable X_i :

1. When the stream reaches position p_i : record $\text{elem}_i \leftarrow \text{stream}[p_i]$ and set $v_i \leftarrow 1$.
2. For each subsequent position $j > p_i$: if $\text{stream}[j] = \text{elem}_i$, increment v_i .

The estimator for F_2 is:

$$\hat{F}_2 = \frac{1}{k} \sum_{i=1}^k n \cdot (2v_i - 1)$$

It can be shown that $\mathbb{E}[n(2v_i - 1)] = F_2$, so the average over all k variables gives an unbiased estimate. The intuition is that frequent elements are more likely to be sampled and to reappear afterwards, contributing proportionally to m_j^2 .

Implementation detail

The k sampled positions are stored in sorted order, so the update loop can break early as soon as it encounters a position beyond the current stream index.

On average only half the variables are active at any point in the stream, so the expected number of iterations per update call is $k/2$. I used `AMS_NUM_VARS = 100`.

Results

k (variables)	Estimate	Relative error
10	9,246,760,000	4.3%
25	9,891,080,000	2.3%
50	9,677,872,000	0.1%
100	9,214,792,000	4.7%
200	9,716,568,000	0.5%

Table 3: AMS on the subsample (200,000 comments, `SEED=1`, exact $F_2^* = 9,667,184,406$).

k (variables)	Estimate	Relative error
10	4,094,686,303,929	30.8%
25	5,649,804,521,452	4.5%
50	6,104,101,436,235	3.2%
100	5,234,470,416,714	11.5%
200	5,269,086,030,059	10.9%

Table 4: AMS on the full dataset (4,986,461 comments, `SEED=1`, exact $F_2^* = 5,916,853,781,669$).

The estimator is non-monotone in k for a single run, consistent with its theoretical variance of $O(F_2^2/k)$. On the subsample errors stay below 5% for all k ; on the full dataset errors reach 10–30% even at large k , reflecting the higher F_2 and wider variance of the larger stream.

4.3 Bloom Filter (Set Membership)

Given the set S of Opinion article IDs, the goal is to answer membership queries “is article $x \in S$?” in $O(1)$ time, tolerating a small rate of false positives.

Data structure

A Bloom filter consists of a bit array of m bits (initially all zero) and k independent hash functions $h_1, \dots, h_k : U \rightarrow [0, m)$.

- **Insertion of x :** set bits $h_1(x), \dots, h_k(x)$ to 1.

- **Query for x :** return TRUE iff all k bits are set.

False negatives are impossible; false positives occur when all k hash positions of a non-member element happen to be set by other insertions. The theoretical false positive rate (FPR) for n inserted elements is:

$$\text{FPR}(m, k, n) = \left(1 - e^{-kn/m}\right)^k$$

The optimal number of hash functions for a given m/n ratio is $k^* = (m/n) \ln 2$.

Bit-array size

The number of hash functions is fixed at `BF_K_HASHES` = $k = 7$. This value is an upper bound on k^* for all targets tested with the full dataset ($n = 2,272$): the most stringent target of $p = 1\%$ gives $k^* \approx 6.65$, and larger targets give smaller k^* .

Rather than fixing m as a global constant, the filter computes it automatically from n , k , and a target FPR p by solving $\text{FPR}(m, k, n) = p$ for m with k fixed:

$$m = \left\lceil \frac{-k n}{\ln(1 - p^{1/k})} \right\rceil$$

The default target is `BF_TARGET_FPR` = 0.01 (1%), giving $m \approx 21,796$ bits for the 2,272 full-dataset Opinion articles.

Hash functions

Each of the k hash functions is implemented as:

$$h_i(x) = \text{MD5}(x \parallel \text{salt}_i) \bmod m$$

where salt_i is a distinct random 32-bit integer string generated at construction time; MD5 works well here since article IDs are URI strings.

Results

Target FPR	m (bits)	FPR empirical	FPR theoretical
50%	6,735	50.40%	50.00%
20%	10,049	19.44%	19.99%
10%	12,505	9.67%	10.00%
5%	15,074	4.99%	5.00%
1%	21,796	1.01%	1.00%

Table 5: Bloom filter on the full dataset ($n = 2,272$ Opinion articles, $k = 7$, 14,515 negative queries, `SEED=1`).

Target FPR	m (bits)	FPR empirical	FPR theoretical
50%	338	60.18%	49.98%
20%	505	19.30%	19.91%
10%	628	9.65%	9.97%
5%	757	4.91%	4.98%
1%	1,094	0.70%	1.00%

Table 6: Bloom filter on the subsample ($n = 114$ Opinion articles, $k = 7$, 570 negative queries, `SEED=1`).

The theoretical FPR matches the target across all settings. On the full dataset the empirical FPR tracks the target closely for all values of p ; on the subsample the 50% target shows a larger deviation (60% vs. 50%) due to the high variance of the observed proportion when both n (114 inserted elements) and the number of negative queries (570) are small.

5 Scalability

Algorithm	Memory (algorithm state)	Time per element
Flajolet-Martin	$O(H)$	$O(H)$
AMS	$O(k)$	$O(k)$
Bloom filter	$O(m)$	$O(k_{\text{BF}})$

Table 7: Complexity of the three algorithms. H = number of hash functions (FM), k = number of AMS variables, m = number of Bloom filter bits, k_{BF} = number of Bloom filter hash functions.

The memory used by the algorithm *state* is constant with respect to the stream length n : FM stores $H = 64$ integers, AMS stores $k = 100$ counters and element labels, and the Bloom filter stores an m -bit array where m is determined by n and the target FPR (e.g. $m \approx 21,796$ bits ≈ 2.7 KB for $n = 2,272$ and target 1%).

References

- [1] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 3rd ed., Cambridge University Press, 2020. <http://www.mmds.org/>
- [2] B. Awad, *New York Times Articles & Comments (2020)*, Kaggle, 2020. <https://www.kaggle.com/datasets/benjaminawd/new-york-times-articles-comments-2020>

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.